Same Signal, Opposite Meaning: Direction-Aware Gating for LLM Agents

Anonymous

Abstract

Test-time compute improves decision quality for LLM agents, but deciding *when* to invoke it remains an open problem. Existing adaptive-gating methods share an implicit assumption: that higher uncertainty consistently indicates when extra computation helps. We show this assumption fails in heterogeneous agent settings: the same signal value that predicts rollout benefit in one environment predicts rollout harm in another. When the assumed direction is wrong, better calibration *worsens* performance by more precisely selecting harmful states. We trace this reversal to two coexisting state types with opposite signal–utility relationships: *information-poor* states, where rollouts cannot compensate for missing evidence, and *decision-difficult* states, where rollouts help explore viable alternatives. Since environments differ in their mixture of these two types, no gate with a fixed signal–utility direction can avoid falling below the no-trigger baseline in at least one environment, making per-environment direction discovery necessary. Motivated by this, we propose DIAL (Direction-Informed Adaptive Learning), which learns environment-specific signal–utility relationships from interaction data. DIAL outperforms all fixed-direction baselines while adding zero per-step overhead at deployment.

1 Introduction

Large language models (LLMs) have rapidly advanced across a wide range of tasks, from question answering and code generation to web navigation and embodied decision making (Yao et al., 2023b; Shinn et al., 2023; Zhou et al., 2024). Yet, many tasks remain beyond the reach of a single forward pass, requiring the model to evaluate alternatives, verify intermediate results, or search over candidate actions (Yao et al., 2023a; Hao et al., 2023; Snell et al., 2024). This has motivated *test-time compute*, where additional computation at inference time improves decision quality. Across multiple benchmarks, test-time optimization allows smaller models to match or exceed the performance of larger ones at lower total cost (Snell et al., 2024; Wang et al., 2026).

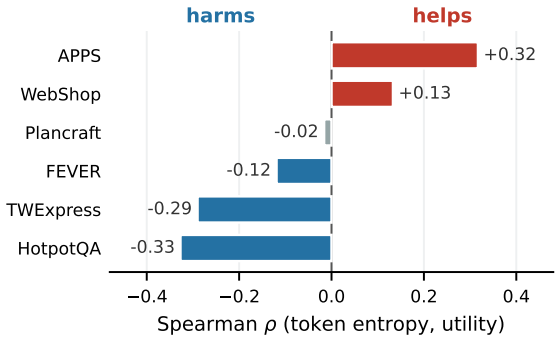

Figure 1: **Same signal, opposite meaning.** Spearman ρ(token entropy, optimizer utility) across six environments on Qwen3-4B. Triggering on high entropy *helps* on APPS and WebShop ($\rho>0$) but *harms* on HotpotQA, TWExpress, and FEVER ($\rho<0$).

While effective, test-time compute is expensive, incurring substantial per-step overhead (e.g., requiring multiple rollouts or candidate evaluations at each decision point) (Yao et al., 2023a; Zhou et al., 2024). As agents are deployed across diverse settings, applying this overhead uniformly becomes wasteful when only a fraction of steps benefit. This has motivated *adaptive gating*: using uncertainty signals to decide *when* extra compute is worthwhile. Prior work explores diverse mechanisms, including entropy thresholding (Lee et al., 2025; Huang et al., 2026; Jin et al., 2026), calibrated confidence (Muennighoff et al., 2025; Han et al., 2025), vote disagreement (Lee et al., 2026), and learned difficulty estimators based on hidden states or reinforcement learning (Liu et al., 2025; Zhu et al., 2025; Zhang et al., 2025; Fang et al., 2025). Implicit in this consensus is a shared as-

sumption, which we call a *fixed-direction* relationship: that higher uncertainty (or lower confidence, higher disagreement) consistently indicates when extra computation helps.

However, this assumption has not been directly tested in heterogeneous agent environments. We show it does not hold: the relationship between uncertainty and compute value can *reverse direction* across environments (Fig. 1), and once this happens, adaptive gating actively underperforms never invoking the optimizer. This raises a central question: *what determines the relationship between uncertainty signals and the value of computation, and can this relationship be learned in a way that generalizes across environments?*

We answer this through systematic measurement across six agent environments spanning QA (Yang et al., 2018), code generation (Hendrycks et al., 2021), web navigation (Yao et al., 2022), fact verification (Thorne et al., 2018), text games (Jansen and Côté, 2023), and crafting planning (Dagan et al., 2024), using three model backbones. The same signal that predicts rollout benefit in code generation predicts rollout harm in fact verification, and can further reverse across model backbones within the same environment. Moreover, the most informative signal itself varies across environments, with structural features dominating in some settings while entropy carries little or no information in others.

We trace this reversal to two coexisting state types with opposite signal–utility relationships. *Information-poor* states (Type I) arise when the agent lacks sufficient information; rollouts from the same model cannot compensate, and high uncertainty predicts *negative* value of computation. In contrast, *decision-difficult* states (Type D) arise when multiple viable options exist; additional computation helps explore alternatives, and high uncertainty predicts *positive* value. For example, code generation is dominated by Type D states (multiple valid implementations exist), while fact verification is dominated by Type I states (evidence is often insufficient). Because environments differ in their mixture of these two types, the same uncertainty signal carries opposite meanings depending on which type dominates, an instance of Simpson's paradox (Simpson, 1951; Wang et al., 2023). The consequence is that no fixed-direction gating strategy can avoid net-harmful gating across environments with opposing signal directions (Proposition 1).

Motivated by this analysis, we propose **DIAL** (**D**irection-**I**nformed **A**daptive **L**earning), which learns environment-specific signal–utility relationships from interaction data. Since the primary challenge is identifying signal direction rather than modeling capacity, DIAL reduces the adaptive-gating problem to a form where a standard sparse linear classifier suffices, adding zero per-step cost at deployment. Across six environments and three LLM backbones, DIAL outperforms all fixed-direction baselines, and its full range of gating behavior, from aggressive triggering to near-total suppression, emerges from direction learning alone.

2 Related Work

We organize prior work by how each method decides when to allocate extra computation.

Gating on uncertainty signals. Signal-based methods allocate extra computation when token confidence, entropy, or disagreement exceeds a threshold. In reasoning settings, this takes the form of extended chain-of-thought generation (Lee et al., 2025; Huang et al., 2026; Jin et al., 2026; Sharma and Chopra, 2025; Muennighoff et al., 2025; Han et al., 2025). In agent settings, vote-based methods (Lee et al., 2026) invoke an arbiter when candidate actions disagree, and verbalized uncertainty methods (Zhang et al., 2026) trigger reflection when self-reported confidence is low.

Gating on learned difficulty. Hidden-state probes (Liu et al., 2025; Zhu et al., 2025) estimate task difficulty from internal representations, and RL-based methods (Zhang et al., 2025; Fang et al., 2025; Aggarwal and Welleck, 2025; Paglieri et al., 2025) learn think/no-think policies end-to-end. Though these methods do not commit to a specific signal, their gating policies effectively assume that higher difficulty warrants more compute, sharing the same fixed direction as signal-based approaches. All have been evaluated on homogeneous reasoning benchmarks (GSM8K (Cobbe et al., 2021), GPQA (Rein et al., 2023)); we extend evaluation to heterogeneous agent environments where this assumption breaks down.

Metareasoning and uncertainty semantics. Rational metareasoning (Russell and Wefald, 1991; Sabbata et al., 2024) formalizes the value of computation (VOC), classically assuming VOC $\geq$

Table 1: Signal–utility correlations across 6 environments and 3 backbones. ρ: Spearman correlation of token entropy with optimizer utility U. *: significant at $p<0.05$. Direction varies across *both* environments and model backbones.

Environment	Qwen3	Phi-3.5	Llama-3.1	Cons.
HotpotQA	-0.327^*	$+0.184^*$	-0.346^*	✗
WebShop	$+0.133^*$	$+0.335^*$	$+0.287^*$	✓
FEVER	-0.119^*	-0.156^*	$+0.428^*$	✗
APPS	$+0.317^*$	-0.024	-0.249^*	✗
TWExpress	-0.290^*	$+0.000$	$+0.000$	—
Plancraft	-0.016	$+0.000$	-0.176^*	—

0 since an agent can disregard unfavorable results. We show this assumption fails when the same model both proposes and evaluates actions (§3.2). Tao et al. (2025) and Heo et al. (2025) find that calibration quality varies across task types, and Abbasi Yadkori et al. (2024) decouple epistemic and aleatoric uncertainty, providing converging evidence that uncertainty semantics are domain-dependent. However, none observes that signal direction can reverse across environments. Our two-source model formalizes this domain dependence and makes it actionable. Extended related work discussion is in Appendix A.

3 Why Fixed-Direction Gating Fails

We first formalize the adaptive gating problem, then measure signal–utility correlations across diverse environments to test the fixed-direction assumption. If the assumption holds, correlations should be consistent; if it fails, the failure mode will guide method design.

We study LLM-based agents in multi-step interactive environments. At each step t, the agent observes state s_t, selects action a_t, and transitions to s_{t+1}. An environment-specific *test-time optimizer* T (e.g., rollout evaluation, K-variant sampling) can be invoked at any step to potentially improve action quality. We define the *optimizer utility* as:

$$U(T, s) = \mathbb{E}[R(\tau) \mid a{=}T(s)] - \mathbb{E}[R(\tau) \mid a{=}\pi(s)] \quad (1)$$

where $R(\tau)$ is the episode return. $U > 0$ indicates the optimizer improves the outcome; $U \leq 0$ indicates it is wasteful or harmful. The *adaptive gating problem* is to learn a gate $g : \mathcal{S} \to \{0, 1\}$ that triggers T only when $U > 0$.

3.1 Signal–Utility Direction Reverses Across Environments and Backbones

The signal–utility direction is not fixed. Table 1 reports Spearman correlations between token entropy and optimizer utility across 6 environments and 3 model backbones (Team, 2025, 2024; Microsoft, 2024). Two patterns stand out. First, *even within a single backbone, the direction varies across environments*: on Qwen3, entropy is negatively correlated with utility in information-seeking environments (FEVER, HotpotQA, TWExpress) but positively in code generation (APPS) and web navigation (WebShop), with near-zero correlation in weak-signal environments (Plancraft). Second, *the same signal in the same environment can reverse across backbones*: in 3 out of 4 environments with multiple significant correlations, models disagree on the sign. Most strikingly, FEVER flips from $\rho{=}-0.156$ on Phi-3.5 to $\rho{=}+0.428$ on Llama-3.1. Crucially, this direction mismatch is not benign: as Section 5.2 will show, gates that assume the wrong direction systematically fall *below* the no-trigger baseline, making them strictly worse than doing nothing in mismatched environments.

Signal identity also varies across environments. Beyond direction, the most informative signal itself differs across environments: `step_count` dominates in most settings, but WebShop relies on `num_available_actions` instead (Appendix F.1). No single signal is reliably informative: single-feature gates are near-chance (AUC≈0.50), while multi-signal gates reach AUC≈0.83 and hidden-state probes reach AUC≈0.90 (Appendix F.3). This motivates the multi-signal, per-environment design of our method (§4).

3.2 Why Does Direction Reverse? A Two-Source Model

The findings above establish that fixed-direction gating fails; we now ask *why*. We trace direction reversal to two coexisting state types with opposite signal–utility relationships.

Intuition. Consider a web-shopping agent choosing between products. When it hesitates (high entropy) because multiple items match the query, exploring alternatives via rollouts is valuable, since the agent has adequate information but faces a genuine choice. Now consider a fact-verification agent that hesitates because the

retrieved evidence is ambiguous. Here, high entropy reflects *missing information*, not optionality: rollouts from the same model with the same evidence will not resolve the ambiguity. In the first case, entropy positively predicts rollout utility; in the second, it predicts rollout *futility*. The same numerical signal carries opposite meaning because the *source* of uncertainty differs.

Model. We model direction reversal as arising from two state types at each decision step:

- **Type I (information poverty):** Utility is *negatively* related to entropy: $U_I(s) \sim -\alpha H(s) + \varepsilon_I$, $\alpha > 0$.
- **Type D (decision difficulty):** Utility is *positively* related to entropy: $U_D(s) \sim +\beta H(s) + \varepsilon_D$, $\beta > 0$.

Let $p_I(\mathcal{E})$ denote the fraction of Type I states in environment $\mathcal{E}$. The marginal correlation becomes

$$\rho(\mathcal{E}) \approx \beta - (\alpha + \beta)\, p_I(\mathcal{E}), \tag{2}$$

which reverses sign at the critical threshold $p_I^* = \beta/(\alpha + \beta)$. Environments with $p_I > p_I^*$ exhibit negative ρ (Type I dominated); those with $p_I < p_I^*$ exhibit positive ρ (Type D dominated); near p_I^*, the two effects cancel and entropy carries negligible signal. This prediction is consistent with the observed Qwen3 pattern in Table 1: search-based QA (FEVER, HotpotQA, TWExpress) shows negative ρ, code generation (APPS) shows positive ρ, and weak-signal environments (Plancraft) cluster near zero. Multi-backbone results (Appendix G.3) further show that the model backbone modulates the effective p_I, making direction unpredictable from environment structure alone.

Theoretical grounding. This direction reversal is a special case of Simpson's paradox (Simpson, 1951; Pearl, 2022): aggregating subpopulations with opposing within-group trends reverses the aggregate correlation. Our decomposition adapts the epistemic/aleatoric distinction (Der Kiureghian and Ditlevsen, 2009) to the meta-decision setting: Type I states exhibit epistemic-like uncertainty (information deficit), while Type D states exhibit aleatoric-like diversity (multiple viable paths). We treat this two-source decomposition as a *minimal falsifiable* model rather than a post-hoc explanation: in Section 5.4 we derive three predictions from it (temporal dynamics, cross-environment consistency, and signal identity alignment) and verify each on held-out data.

3.3 Necessity of Direction Discovery

Proposition 1 (Necessity of Direction Discovery)**.** *Suppose there exist two environments $\mathcal{E}_1, \mathcal{E}_2$ with opposite true directions: $d^*(\mathcal{E}_1) = +1$ and $d^*(\mathcal{E}_2) = -1$. Then no fixed-direction gate g_d can simultaneously satisfy $\mathrm{SR}(g_d, \mathcal{E}_1) \geq \mathrm{SR}(\mathrm{base}, \mathcal{E}_1)$ and $\mathrm{SR}(g_d, \mathcal{E}_2) \geq \mathrm{SR}(\mathrm{base}, \mathcal{E}_2)$.*

The intuition is direct: a gate with $d = +1$ triggers on high-signal states, which in a Type I environment correspond exactly to states where rollouts harm the agent; the symmetric argument holds for $d = -1$ in Type D environments. Since real-world deployments span both environment types, no fixed-direction strategy can deliver non-negative value of computation universally. Direction discovery is therefore a necessary precondition for cross-environment robustness, not a refinement of it. The full proof is in Appendix B.1.

4 Direction-Informed Adaptive Learning

Section 3 imposes two requirements on any gating method that aims for cross-environment robustness: signal direction must be *learned* per environment (Proposition 1), and the feature set must be *multi-signal* (§3.1). A third follows from the method design: exploration must be *signal-agnostic*, to avoid biasing data collection toward the unknown quantity being estimated.

DIAL (*Direction-Informed Adaptive Learning*) learns the correct signal interpretation for each environment through a lightweight explore–reason–learn pipeline (Fig. 2): randomized exploration (§4.1) handles the unknown prior direction, an optional LLM feature layer (§4.2) proposes task-specific signals when universal ones are inadequate, and a sparse linear gate (§4.3) recovers per-environment direction from the resulting dataset.

4.1 Explore: Signal-Agnostic Data Collection

Exploration collects the (s, U) pairs that will be used to fit the gate. The central constraint is signal-agnosticity: the collection policy must not condition on any signal whose semantics we are trying to learn, as this would bias $\mathcal{D}$ toward the unknown quantity being estimated. DIAL therefore uses the simplest policy consistent with this constraint. At each step t in episode i, a uniform random coin flip $z_t^{(i)} \sim \mathrm{Bernoulli}(\varepsilon_{\text{explore}})$ determines whether to trigger the optimizer T (z_t=1) or use the base action (z_t=0). Aggregating across N_{explore} episodes

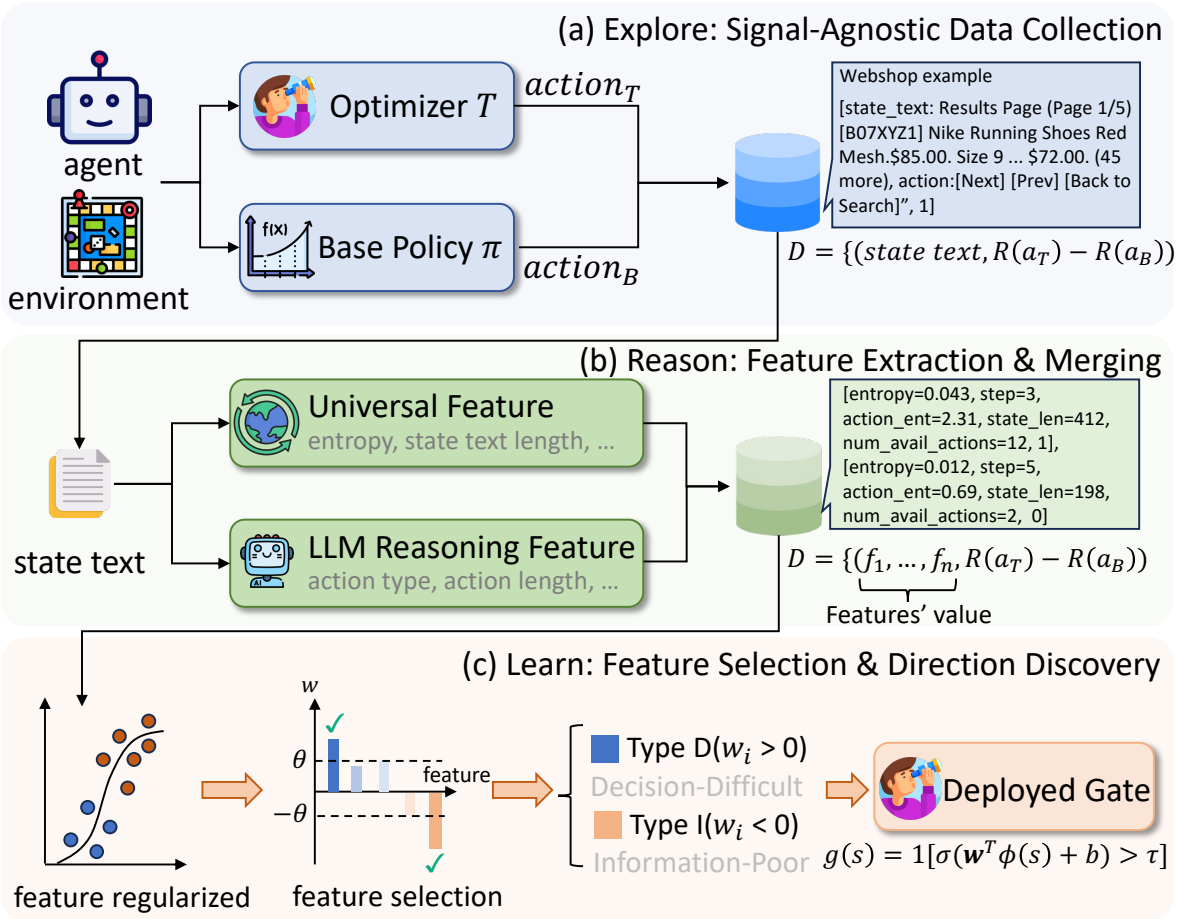

Figure 2: **The DIAL pipeline.** **(a) Explore**: collect signal-agnostic (s, U) pairs via Bernoulli-ε rollouts. **(b) Reason**: build a candidate feature pool $\phi(s)$ from universal metrics plus optional LLM-proposed task-specific features. **(c) Learn**: fit an ℓ_1-regularized logistic gate, whose signed weights jointly perform feature selection and recover per-environment direction (Type D: $w_i{>}0$; Type I: $w_i{<}0$).

yields an unbiased dataset:

$$\mathcal{D} = \left\{ \left(\phi(s_t^{(i)}),\ U_t^{(i)} \right) : z_t^{(i)} = 1 \right\}_{t,i}, \qquad (3)$$

where $U_t^{(i)} \in \{0, 1\}$ indicates whether the optimizer improved the outcome at step t, and $\phi(s)$ is the feature vector described in §4.2.

4.2 Reason: LLM Feature Layer

The next step builds the feature pool $\phi_{\text{cand}}(s)$ that the gate will be fit over. This pool must be rich enough to include each environment's relevant signals, but not so environment-specific that deployment requires manual engineering per task. DIAL's default is a fixed set of universal features: `step_count`, `token_entropy`, `state_length`, `action_entropy`, and others. These are available in every environment without task-specific engineering. In the environments we study, this pool alone is sufficient to recover a gate that outperforms all fixed-direction baselines (§5.3).

For environments where universal features may be inadequate, such as WebShop, whose gating benefits from task-specific signals like `num_available_actions`, DIAL offers an *optional* automation layer: an LLM is given a summary of $\mathcal{D}$ and proposes task-specific feature hypotheses $\phi_{\text{LLM}}(s)$, which join the universal pool to form $\phi_{\text{cand}}(s) = \phi_{\text{univ}}(s) \cup \phi_{\text{LLM}}(s)$. Details of the LLM input format and prompt template are in Appendix C.3. The ablation in §5.3 shows its gains concentrate in the few settings where universal features miss the relevant structure, so the LLM is best viewed as an automation convenience for zero-shot deployment rather than a core component the method depends on.

4.3 Learn: Sparse Linear Gate

Given $\mathcal{D}$ and ϕ_{cand}, DIAL fits an ℓ_1-regularized logistic regression:

$$\mathbf{w}^*, b^* = \arg\min_{\mathbf{w},b} \sum_{(\phi,U)\in\mathcal{D}} \ell\big(\sigma(\mathbf{w}^\top\phi + b),\, U\big) + \lambda\|\mathbf{w}\|_1 \tag{4}$$

where ℓ is the binary cross-entropy loss and λ controls sparsity over ϕ_{cand} so that uninformative features drop out. The deployed gate is

$$g(s) = \mathbf{1}\Big[\sigma(\mathbf{w}^{*\top}\phi(s) + b^*) > \tau\Big], \tag{5}$$

with threshold τ set by cross-validation on $\mathcal{D}$. Gate fitting completes in under a second, and deployment adds zero per-step overhead beyond feature computation. Implementation details are in Appendix C.

Signed weights as a diagnostic. The ℓ_1 penalty in Eq. 4 simultaneously performs feature selection and direction discovery. The sign of each fitted weight provides an interpretable per-environment estimate of the Two-Source mixture of §3.2: $w_i > 0$ indicates the environment sits on the Type D side of the reversal threshold p_I^* for signal i (high signal $\rightarrow$ trigger), $w_i < 0$ the Type I side (high signal $\rightarrow$ *don't* trigger), and $w_i \approx 0$ an uninformative signal. The complete pipeline, including online adaptation for drifting environments, is summarized in Algorithm 1 (Appendix C).

5 Experiments

5.1 Experimental Setup

We evaluate DIAL along three axes: does environment-aware gating Pareto-dominate fixed-direction baselines (§5.2); which components drive the gains (§5.3); and do the Two-Source Model's predictions hold on held-out data (§5.4).

Environments. We evaluate on 6 environments: HotpotQA (Yang et al., 2018), APPS (Hendrycks et al., 2021), WebShop (Yao et al., 2022), FEVER (Thorne et al., 2018), TWExpress (Jansen and Côté, 2023), and Plancraft (Dagan et al., 2024). These cover the full range of the Two-Source Model, from pure Type I (FEVER) through mixed (WebShop) to Type D (APPS), including extreme rollout properties (TWExpress: rollout-safe; Plancraft: rollout-harmful). Full environment specifications are in Appendix D.1.

Baselines. All methods share the same agent and optimizer T; only the gating decision differs. We include two bounds: *base_only*, which never triggers the optimizer, and *always_trigger*. Six fixed-direction baselines all assume that high uncertainty warrants additional compute: CaTS (Huang et al., 2026), SEAG (Lee et al., 2025), CoRefine (Jin et al., 2026), CATTS (Lee et al., 2026), AUQ (Zhang et al., 2026), and s1_budget (Muennighoff et al., 2025). Full specifications are in Appendix D.2.

Metrics. SR (success rate, ↑) and Cost (×base, ↓): total deployment tokens normalized by base_only tokens per episode. This metric counts *all* LLM calls, including base proposer, gate overhead (CATTS K=5 voting, AUQ confidence queries), and rollout calls, ensuring that per-step overhead is not hidden. Calibration or exploration overhead is excluded for all methods. A method Pareto-dominates another if SR $\geq$ and Cost $\leq$ with at least one strict inequality. Token cost constants are reported in Appendix D.3.

5.2 Main Results

DIAL Pareto-dominates fixed-direction baselines. Figure 3 shows the SR–Cost Pareto frontier across all 6 environments on Qwen3-4B. Full results across three backbones are in Appendix E.

Three patterns emerge. First, *fixed-direction baselines catastrophically fail when the assumed direction is wrong*: on FEVER (Type I), CATTS achieves only 32.2% SR (below the 37.0% no-trigger baseline) yet costs 16.46×base due to its K=5 per-step voting overhead. Second, *DIAL achieves competitive SR at substantially lower cost*. On WebShop, DIAL reaches 43.8% SR at 2.50×base, matching always_trigger (43.0%) at less than half the cost (5.56×). On APPS, DIAL outperforms the best baseline (s1_budget, 69.0%) by 4 pp at comparable cost. Third, *in rollout-harmful environments, DIAL learns to suppress triggering*: in Plancraft (Δ=−7.0), DIAL recovers near base_only performance (23.3% vs. 29.8%), while most baselines that trigger aggressively fall below base_only (s1_budget: 16.3%).

DIAL adapts its behavior to each environment. Table 2 compares DIAL against the two bounds on Qwen3-4B. In environments with positive rollout headroom (HotpotQA, WebShop, TWExpress, APPS), DIAL achieves SR close to always_trigger at substantially lower cost. In the

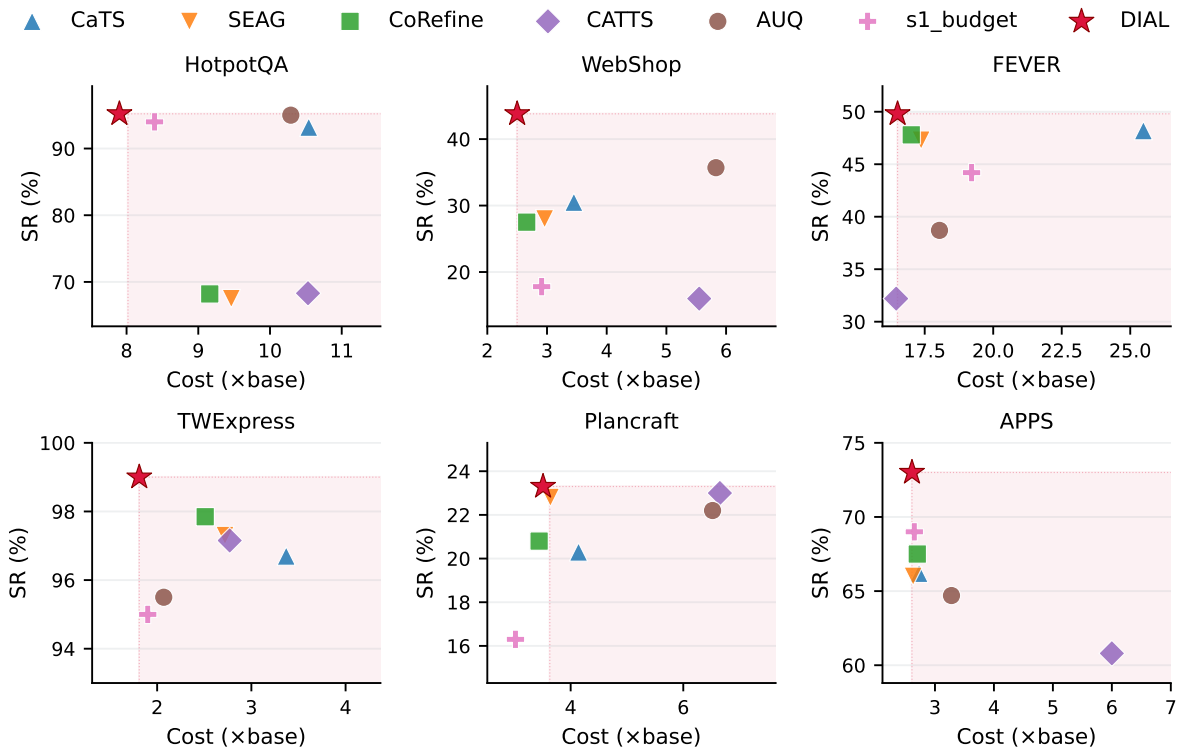

Figure 3: SR vs. Cost (×base) across all 6 environments (Qwen3-4B). Shaded region: dominated by DIAL (lower SR *and* higher cost). Full numerical results across three backbones are in Table 10.

Table 2: DIAL vs. bounds on Qwen3-4B. Δ: rollout headroom (always−base).

	base_only		**always_trigger**		**DIAL**	
Env	SR↑	Cost↓	SR↑	Cost↓	SR↑	Cost↓
HotpotQA	49.0	1.00	97.0	10.63	95.2	8.02
WebShop	7.2	1.00	43.0	5.56	43.8	2.50
FEVER	37.0	1.00	59.8	17.87	49.8	16.51
TWExpress	67.5	1.00	99.3	2.15	99.0	1.81
Plancraft	29.8	1.00	22.8	6.13	23.3	3.63
APPS	60.5	1.00	79.5	3.65	73.0	2.61

Table 3: Effect of reversing DIAL's learned direction. ΔSR: reversed − original. Degradation scales with signal strength $|\rho|$.

Environment	DIAL SR	Reversed SR	Δ SR	$\lvert\rho\rvert$
FEVER	49.8%	13.0%	−36.8	0.619
HotpotQA	95.2%	58.2%	−37.0	0.494
WebShop	43.8%	20.6%	−23.2	0.444
APPS	73.0%	70.5%	−2.5	0.339
TWExpress	99.0%	99.0%	0.0	0.290
Plancraft	23.3%	20.5%	−2.8	0.016

rollout-harmful environment Plancraft (Δ=−7.0), DIAL learns to suppress triggering, losing only 6.5 pp relative to base_only while always_trigger loses 7.0 pp. FEVER remains the weakest case: DIAL improves over base_only (49.8% vs. 37.0%) but falls far short of always_trigger (59.8%), a limitation analyzed in §6.

5.3 Ablation Studies

The main results show *that* DIAL works; we now ask *why*. We ablate the two key components: direction learning and the LLM feature layer.

Direction is the primary determinant of gate quality. To isolate the effect of direction, we reverse the sign of all learned weights in DIAL's fitted gate, keeping everything else unchanged. Table 3 shows the result. In environments with strong signal ($|\rho| > 0.4$), reversal causes SR drops exceeding 23 pp. In weak-signal environments like Plancraft ($|\rho|$=0.016), reversal has negligible effect. The degradation scales with signal strength: wrong direction is not a minor calibration error but a structural failure, as Proposition 1 predicts.

The LLM feature layer is an automation convenience. Table 4 compares DIAL with and without LLM-generated features. FEVER is the only environment where LLM features contribute meaningfully (+9.1 pp), where task-specific signals (`has_claim`, `text_length`) extend the universal feature pool. In the remaining environ-

Table 4: LLM feature ablation. SR (%) with and without LLM-generated features. FEVER is the only environment with meaningful LLM contribution.

Environment	w/ LLM	w/o LLM	Δ
HotpotQA	95.2	93.8	+1.4
WebShop	43.8	42.7	+1.1
FEVER	49.8	40.7	+9.1
TWExpress	99.0	97.2	+1.8
Plancraft	23.3	23.4	−0.2
APPS	73.0	73.0	+0.0

ments, removing LLM features changes SR by <2 pp; in Plancraft and APPS the contribution is effectively zero.

ℓ_1 is not interchangeable with other shrinkage or selection schemes. The signed-weight diagnostic of §4.3 and DIAL's tolerance of noisy LLM-proposed features both rely on the *exact* sparsity induced by ℓ_1. Replacing it with ℓ_2 yields comparable SR on strong-signal environments but eliminates the clean {informative, uninformative} partition, because every feature retains a small non-zero weight; unregularized logistic overfits on weak-signal environments like Plancraft; hard mutual-information feature selection recovers SR on strong-signal tasks but requires per-environment tuning of k, reintroducing the engineering cost DIAL aims to eliminate. The full comparison across regularizers, holding $\mathcal{D}$ and ϕ_{cand} fixed, is reported in §G.5.

5.4 Two-Source Model Verification

The ablations confirm DIAL's practical advantage; we now test whether the Two-Source Model (§3.2) has predictive power beyond explaining the observed correlations in Table 1. Following Schaeffer et al. (2023), we derive three testable predictions and verify each.

P1: Temporal dynamics. The Two-Source Model predicts that ρ should decrease over the episode: as the agent gathers information, Type D states are resolved and late-step uncertainty increasingly reflects residual Type I states. Figure 4 confirms this. In HotpotQA, ρ shifts from -0.176 (early) to -0.437 (late). In WebShop, the strong positive early-phase signal (ρ=+0.285) vanishes in the late phase. In APPS, ρ reverses from positive (+0.102) to negative (−0.144). Full numerical results are in Appendix G.1.

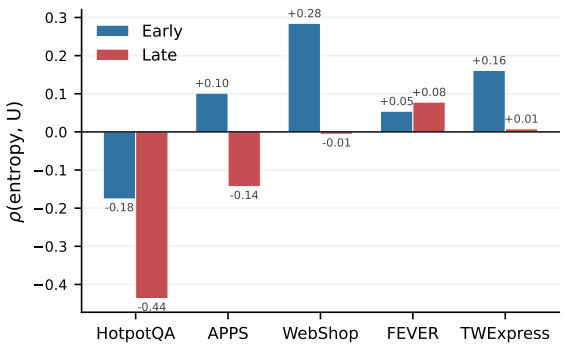

Figure 4: P1 verification: ρ(entropy, U) shifts from early to late steps. Blue: early (step $\leq$ median), red: late. Values on each bar show the Spearman ρ for that phase. Plancraft is omitted ($|\rho|$=0.016, too weak for meaningful comparison).

P2: Cross-environment consistency. The Two-Source Model predicts that environments with similar task structure should exhibit similar ρ. On Qwen3 (Table 1), this holds: FEVER and HotpotQA (both search-based QA) share negative ρ; WebShop and APPS (both involving choice among alternatives) share positive ρ. This consistency does not hold across all backbones, as the model backbone modulates the effective Type I fraction and can override task-structure similarity (Appendix G.3).

P3: Signal identity alignment. The Two-Source Model predicts that the strongest signal should reflect the dominant state type: information-sufficiency proxies in Type I environments, decision-complexity proxies in Type D environments. Appendix G.2 confirms this: `step_count` dominates in Type I environments, while `num_available_actions` dominates in WebShop.

To provide causal evidence, we manipulate information availability within HotpotQA itself. Table 5 shows the results: restricting evidence (InfoPoor) makes `step_count` the dominant signal (ρ=−0.608) while entropy is weak; injecting gold evidence (InfoRich) makes entropy dominant (ρ=+0.311) while `step_count` weakens. This confirms that signal identity is a function of information structure, not an environment-specific constant. DIAL adapts to both conditions without manual reconfiguration.

All three predictions are supported. Additional robustness checks are in Appendix G: multi-backbone verification (§G.3), stratified analysis

Table 5: Controlled information manipulation on HotpotQA. Manipulating information availability shifts which signal dominates, confirming the Two-Source Model's prediction.

Condition	Base SR	ρ(entropy)	ρ(step_count)	DIAL SR	n
Original	49.0%	-0.041	-0.494	95.2%	1117
InfoPoor	44.0%	$+0.119$	-0.608	94.0%	317
InfoRich	82.0%	$+0.311$	-0.147	88.3%	221

(§G.4), and gate complexity ablations (§G.6).

6 Limitations

DIAL's weakest result is on FEVER, where all methods incur high cost due to the environment's extreme Type I structure: DIAL costs 16.51×base and achieves 49.8% SR, while cheaper alternatives like SEAG (47.3% at 17.19×) are only marginally lower in both SR and cost. Rollout value in FEVER concentrates in the first one or two steps, and random exploration misses these critical steps with 50% probability. This failure mode is predicted by the Two-Source Model: FEVER is the most extreme Type I environment, where undirected exploration is least efficient. Addressing this through curriculum-based exploration is an important direction for future work. We verify our findings on three backbones (4B–8B) across all six environments (Appendix G.3); testing on larger models (70B+) remains future work. The Two-Source Model assumes linear conditional relationships and serves as an explanatory framework rather than a precise predictor of ρ values. DIAL requires 50 exploration episodes per new environment, a modest but non-zero cost amortized over subsequent deployment.

7 Conclusion

Existing adaptive gating methods for LLM agents assume that uncertainty signals have a fixed relationship with compute value. We show this assumption is a structural limitation: uncertainty signals are not sufficient statistics for the value of computation, because their meaning depends on a latent state property (information poverty vs. decision difficulty) that the signal alone cannot identify. Across six environments and three model backbones, signal–utility direction reverses, and once misaligned, better calibration *worsens* performance by more precisely selecting harmful states. A two-source model grounded in Simpson's paradox explains when and why reversal occurs, and a necessity proof establishes that direction discovery is unavoidable for cross-environment robustness. DIAL instantiates this principle with a minimal pipeline that recovers per-environment direction through sparse linear learning, adding zero per-step overhead at deployment. Across all evaluated settings, DIAL dominates fixed-direction baselines, and its full range of gating behavior emerges from direction learning alone.

References

Yasin Abbasi Yadkori, Ilja Kuzborskij, and András György. 2024. To believe or not to believe your LLM. *CoRR*.

Pranjal Aggarwal and Sean Welleck. 2025. L1: controlling how long A reasoning model thinks with reinforcement learning. *CoRR*.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. 2021. Training verifiers to solve math word problems. *CoRR*.

Gautier Dagan, Frank Keller, and Alex Lascarides. 2024. Plancraft: an evaluation dataset for planning with LLM agents. *CoRR*.

Armen Der Kiureghian and Ove Ditlevsen. 2009. Aleatory or epistemic? does it matter? *Structural safety*.

Guanting Dong, Hangyu Mao, Kai Ma, Licheng Bao, Yifei Chen, Zhongyuan Wang, Zhongxia Chen, Jiazhen Du, Huiyang Wang, Fuzheng Zhang, Guorui Zhou, Yutao Zhu, Ji-Rong Wen, and Zhicheng Dou. 2025. Agentic reinforced policy optimization. *CoRR*.

Mostafa Elhoushi, Akshat Shrivastava, Diana Liskovich, Basil Hosmer, Bram Wasti, Liangzhen Lai, Anas Mahmoud, Bilge Acun, Saurabh Agarwal, Ahmed Roman, Ahmed A Aly, Beidi Chen, and Carole-Jean Wu. 2024. Layerskip: Enabling early exit inference and self-speculative decoding. In *ACL*.

Gongfan Fang, Xinyin Ma, and Xinchao Wang. 2025. Thinkless: LLM learns when to think. *CoRR*.

Alex Graves. 2016. Adaptive computation time for recurrent neural networks. *CoRR*.

Tingxu Han, Zhenting Wang, Chunrong Fang, Shiyu Zhao, Shiqing Ma, and Zhenyu Chen. 2025. Token-budget-aware LLM reasoning. In *ACL*.

Shibo Hao, Yi Gu, Haodi Ma, Joshua Jiahua Hong, Zhen Wang, Daisy Zhe Wang, and Zhiting Hu. 2023. Reasoning with language model is planning with world model. In *EMNLP*.

Dan Hendrycks, Steven Basart, Saurav Kadavath, Mantas Mazeika, Andy Zou, Dawn Song, and Jacob Steinhardt. 2021. Measuring coding challenge competence with APPS. In *NeurIPS*.

Juyeon Heo, Miao Xiong, Christina Heinze-Deml, and Jaya Narain. 2025. Do llms estimate uncertainty well in instruction-following? In *ICLR*.

Eric Horvitz. 1987. Reasoning about beliefs and actions under computational resource constraints. In *UAI*.

Chengsong Huang, Langlin Huang, Jixuan Leng, Jiacheng Liu, and Jiaxin Huang. 2026. Cats: Calibrated test-time scaling for efficient llm reasoning. In *ICLR*.

Peter Jansen and Marc-Alexandre Côté. 2023. Textworldexpress: Simulating text games at one million steps per second. In *EACL*.

Chen Jin, Ryutaro Tanno, Tom Diethe, and Philip Teare. 2026. Corefine: Confidence-guided self-refinement for adaptive test-time compute. *CoRR*.

Saurav Kadavath, Tom Conerly, Amanda Askell, Tom Henighan, Dawn Drain, Ethan Perez, Nicholas Schiefer, Zac Hatfield-Dodds, Nova DasSarma, Eli Tran-Johnson, Scott Johnston, Sheer El Showk, Andy Jones, Nelson Elhage, Tristan Hume, Anna Chen, Yuntao Bai, Sam Bowman, Stanislav Fort, and 17 others. 2022. Language models (mostly) know what they know. *CoRR*.

Nicholas Lee, Lutfi Eren Erdogan, Chris Joseph John, Surya Krishnapillai, Michael W. Mahoney, Kurt Keutzer, and Amir Gholami. 2026. Agentic test-time scaling for webagents. *CoRR*.

Sungjae Lee, Hyejin Park, Jaechang Kim, and Jungseul Ok. 2025. Semantic exploration with adaptive gating for efficient problem solving with language models. In *ACL*.

Falk Lieder and Thomas L Griffiths. 2020. Resource-rational analysis: Understanding human cognition as the optimal use of limited computational resources. *Behavioral and brain sciences*.

Xiang Liu, Xuming Hu, Xiaowen Chu, and Eunsol Choi. 2025. Diffadapt: Difficulty-adaptive reasoning for token-efficient LLM inference. *CoRR*.

Microsoft. 2024. Phi-3 technical report: A highly capable language model locally on your phone. *CoRR*.

Niklas Muennighoff, Zitong Yang, Weijia Shi, Xiang Lisa Li, Li Fei-Fei, Hannaneh Hajishirzi, Luke Zettlemoyer, Percy Liang, Emmanuel J. Candès, and Tatsunori Hashimoto. 2025. s1: Simple test-time scaling. In *EMNLP*.

Davide Paglieri, Bartlomiej Cupial, Jonathan Cook, Ulyana Piterbarg, Jens Tuyls, Edward Grefenstette, Jakob Nicolaus Foerster, Jack Parker-Holder, and Tim Rocktäschel. 2025. Learning when to plan: Efficiently allocating test-time compute for LLM agents. *CoRR*.

Judea Pearl. 2022. Comment: Understanding simpson's paradox. In *Probabilistic and Causal Inference: The Works of Judea Pearl*.

David Raposo, Samuel Ritter, Blake A. Richards, Timothy P. Lillicrap, Peter Conway Humphreys, and Adam Santoro. 2024. Mixture-of-depths: Dynamically allocating compute in transformer-based language models. *CoRR*.

David Rein, Betty Li Hou, Asa Cooper Stickland, Jackson Petty, Richard Yuanzhe Pang, Julien Dirani, Julian Michael, and Samuel R. Bowman. 2023. GPQA: A graduate-level google-proof q&a benchmark. *CoRR*.

Stuart J. Russell and Eric Wefald. 1991. *Do the right thing - studies in limited rationality*. MIT Press.

Nicolò De Sabbata, Theodore R. Sumers, and Thomas L. Griffiths. 2024. Rational metareasoning for large language models. *CoRR*.

Rylan Schaeffer, Brando Miranda, and Sanmi Koyejo. 2023. Are emergent abilities of large language models a mirage? In *NeurIPS*.

Aman Sharma and Paras Chopra. 2025. Think just enough: Sequence-level entropy as a confidence signal for LLM reasoning. *CoRR*.

Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc V. Le, Geoffrey E. Hinton, and Jeff Dean. 2017. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *ICLR*.

Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: language agents with verbal reinforcement learning. In *NeurIPS*.

Edward H Simpson. 1951. The interpretation of interaction in contingency tables. *Journal of the Royal Statistical Society*.

Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. 2024. Scaling LLM test-time compute optimally can be more effective than scaling model parameters. *CoRR*.

Linwei Tao, Yi-Fan Yeh, Minjing Dong, Tao Huang, Philip Torr, and Chang Xu. 2025. Revisiting uncertainty estimation and calibration of large language models. *CoRR*.

Llama Team. 2024. The llama 3 herd of models. *CoRR*.

Qwen Team. 2025. Qwen3 technical report. *CoRR*.

Surat Teerapittayanon, Bradley McDanel, and H. T. Kung. 2017. Branchynet: Fast inference via early exiting from deep neural networks. *CoRR*.

James Thorne, Andreas Vlachos, Christos Christodoulopoulos, and Arpit Mittal. 2018. Fever: a large-scale dataset for fact extraction and verification. In *NAACL*.

Jingwei Wang, Jianshan He, Weidi Xu, Ruopeng Li, and Wei Chu. 2023. Learning to discover various simpson's paradoxes. In *KDD*.

Zehong Wang, Fang Wu, Hongru Wang, Xiangru Tang, Bolian Li, Zhenfei Yin, Yijun Ma, Yiyang Li, Weixiang Sun, Xiusi Chen, and Yanfang Ye. 2026. Why reasoning fails to plan: A planning-centric analysis of long-horizon decision making in LLM agents. *CoRR*.

Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W. Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. 2018. Hotpotqa: A dataset for diverse, explainable multi-hop question answering. In *EMNLP*.

Shunyu Yao, Howard Chen, John Yang, and Karthik Narasimhan. 2022. Webshop: Towards scalable real-world web interaction with grounded language agents. In *NeurIPS*.

Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Tom Griffiths, Yuan Cao, and Karthik Narasimhan. 2023a. Tree of thoughts: Deliberate problem solving with large language models. In *NeurIPS*.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R. Narasimhan, and Yuan Cao. 2023b. React: Synergizing reasoning and acting in language models. In *ICLR*.

Jiajie Zhang, Nianyi Lin, Lei Hou, Ling Feng, and Juanzi Li. 2025. Adaptthink: Reasoning models can learn when to think. In *EMNLP*.

Jiaxin Zhang, Prafulla Kumar Choubey, Kung-Hsiang Huang, Caiming Xiong, and Chien-Sheng Wu. 2026. Agentic uncertainty quantification. *CoRR*.

Andy Zhou, Kai Yan, Michal Shlapentokh-Rothman, Haohan Wang, and Yu-Xiong Wang. 2024. Language agent tree search unifies reasoning, acting, and planning in language models. In *ICML*.

Yubo Zhu, Dongrui Liu, Zecheng Lin, Wei Tong, Sheng Zhong, and Jing Shao. 2025. The LLM already knows: Estimating llm-perceived question difficulty via hidden representations. In *EMNLP*.

A Extended Related Work

A.1 Adaptive Compute: Reasoning vs. Agent Settings

The adaptive test-time compute literature spans two distinct settings. In the *reasoning* setting, methods optimize single-turn chain-of-thought generation on homogeneous benchmarks (MATH, GSM8K, GPQA). Signal-based approaches use token-level confidence or entropy to decide when to extend thinking (Lee et al., 2025; Huang et al., 2026; Jin et al., 2026; Sharma and Chopra, 2025; Muennighoff et al., 2025; Han et al., 2025). RL-based methods learn think/no-think policies through reinforcement learning (Zhang et al., 2025; Fang et al., 2025; Aggarwal and Welleck, 2025). Hidden-state probing methods train lightweight classifiers on internal representations to estimate task difficulty (Liu et al., 2025; Zhu et al., 2025), sometimes assuming a universal U-shaped entropy pattern that our cross-environment evidence disproves. Because reasoning benchmarks share similar task semantics, signal direction happens to be consistent across tasks, and the fixed-direction assumption is never tested.

In the *agent* setting, methods operate in multi-step interactive environments with external optimizers. Vote-based methods (Lee et al., 2026; Jin et al., 2026) sample multiple candidate actions and trigger evaluation when disagreement is high. Verbalized-uncertainty methods (Zhang et al., 2026) use self-reported confidence for agent reflection but assume fixed direction. Episode-level approaches (Paglieri et al., 2025) learn when to invoke a planner, and agentic RL methods (Dong et al., 2025) use entropy-based adaptive rollout at tool-call steps. All agent-setting methods inherit the fixed-direction assumption from the reasoning literature without verifying it. Our work is the first to systematically study signal semantics across heterogeneous agent environments, where we find that the direction reverses.

A.2 Adaptive Computation in Neural Networks

The idea of adapting computation to input difficulty predates LLMs. Adaptive Computation Time (Graves, 2016) allows RNNs to learn how many computational steps to take per input. Early exit methods (Teerapittayanon et al., 2017; Elhoushi et al., 2024) attach classifiers at intermediate transformer layers and exit early for easy inputs. Mixture-of-Experts architectures (Shazeer et al., 2017; Raposo et al., 2024) activate only a subset of parameters per token, adapting capacity rather than depth. These methods adapt computation within a single forward pass. Our work operates at a different granularity: we adapt whether to invoke an external optimizer at each decision step, and the key challenge is not input difficulty but the *direction* of the signal–utility relationship.

A.3 LLM Confidence Estimation and Calibration

Kadavath et al. (2022) show that larger models are well-calibrated on multiple-choice tasks and can estimate P(True) for their own outputs. Tao et al. (2025) and Heo et al. (2025) find that calibration quality varies substantially across task types, providing converging evidence that uncertainty semantics are domain-dependent. Abbasi Yadkori et al. (2024) propose decoupling epistemic and aleatoric uncertainty in LLM outputs, a distinction conceptually related to our Type I/Type D decomposition. Our work extends these findings: not only does calibration *quality* vary across tasks, but the *direction* of the uncertainty–utility relationship reverses entirely.

A.4 Metareasoning and Value of Computation

Rational metareasoning (Russell and Wefald, 1991; Horvitz, 1987) formalizes the value of computation (VOC) as a guide for allocating cognitive resources. The classical framework assumes VOC ≥ 0 because an agent can always ignore computation results. Lieder and Griffiths (2020) generalize this to resource-rational analysis, modeling cognition as optimal use of limited computational resources. Sabbata et al. (2024) apply metareasoning to LLMs, analyzing when to extend chain-of-thought reasoning. Our work shows that the classical VOC ≥ 0 assumption is violated in the agent setting; we formalize this in Appendix B.3.

A.5 Simpson's Paradox in Machine Learning

Simpson's paradox (Simpson, 1951; Pearl, 2022) occurs when a trend present in subgroups reverses upon aggregation. In machine learning, this phenomenon has been documented in fairness (Wang et al., 2023), causal inference, and treatment effect estimation. Our Two-Source Model identifies a new instance: within Type I states, en-

tropy negatively predicts utility; within Type D states, entropy positively predicts utility; but the marginal correlation depends on the mixture proportion $p_I(\mathcal{E})$, which varies across environments. This makes direction reversal a structural consequence of aggregating heterogeneous uncertainty sources.

A.6 Search-Based Optimizers

Search methods such as Tree of Thoughts (Yao et al., 2023a), LATS (Zhou et al., 2024), RAP (Hao et al., 2023), and FLARE (Wang et al., 2026) improve *what* the optimizer does through tree search, value propagation, and lookahead evaluation. These are orthogonal to our work: we control *when* to invoke the optimizer, and our gate composes with any such procedure.

B Theoretical Material

This appendix collects the theoretical material referenced in Section 3: the proof of Proposition 1, the wrong-direction damage quantification, and the VOC negativity analysis.

B.1 Proof of Proposition 1 (Necessity of Direction Discovery)

Proposition (Restated)**.** *Suppose there exist two environments $\mathcal{E}_1, \mathcal{E}_2$ with opposite true directions: $d^*(\mathcal{E}_1) = +1$ and $d^*(\mathcal{E}_2) = -1$. Then no fixed-direction gate g_d can simultaneously satisfy $\mathrm{SR}(g_d, \mathcal{E}_1) \geq \mathrm{SR}(\mathrm{base}, \mathcal{E}_1)$ and $\mathrm{SR}(g_d, \mathcal{E}_2) \geq \mathrm{SR}(\mathrm{base}, \mathcal{E}_2)$.*

Proof. Define a fixed-direction gate as $g_d(s) = \mathbf{1}[d \cdot \sigma(s) > \theta]$ for $d \in \{+1, -1\}$ and threshold θ.

Define $d^*(\mathcal{E}) = +1$ if $\mathrm{Corr}(\sigma(s), U(T, s)) > 0$ in $\mathcal{E}$ (Type D), and $d^*(\mathcal{E}) = -1$ if $\mathrm{Corr}(\sigma(s), U(T, s)) < 0$ (Type I).

Lemma (Wrong-direction harm). If $d \neq d^*(\mathcal{E})$, then $\mathrm{SR}(g_d, \mathcal{E}) \leq \mathrm{SR}(\mathrm{base}, \mathcal{E}) - \delta(\mathcal{E})$ where $\delta(\mathcal{E}) > 0$ depends on signal strength $|\rho(\mathcal{E})|$.

Proof of Lemma: By definition of d^*, states with high $d \cdot \sigma(s)$ are states where $U < 0$. The gate triggers T at these states, causing the agent to adopt worse actions. The base policy, which never triggers T, avoids this systematic harm.

Main argument:

Case 1: $d = +1$. In $\mathcal{E}_2$ where $d^* = -1$, $d \neq d^*(\mathcal{E}_2)$. By Lemma: $\mathrm{SR}(g_{+1}, \mathcal{E}_2) \leq \mathrm{SR}(\mathrm{base}, \mathcal{E}_2) - \delta(\mathcal{E}_2) < \mathrm{SR}(\mathrm{base}, \mathcal{E}_2)$. Constraint violated.

Case 2: $d = -1$. In $\mathcal{E}_1$ where $d^* = +1$, by symmetric argument, constraint violated.

Since $d \in \{+1, -1\}$ and both cases lead to violation, no fixed d satisfies both constraints simultaneously. □

B.2 Wrong-Direction Damage: Supplementary Analysis

Table 3 shows that wrong-direction damage scales with signal strength. Here we discuss two environments where reversal has minimal effect.

TWExpress is rollout-safe (Δ=+31.8): the optimizer rarely harms performance regardless of when it is triggered, so both directions achieve near-perfect SR (99.0%).

APPS shows a small negative ΔSR (-2.5 pp), consistent with its unstable signal direction across backbones (Table 1): Qwen3 shows positive ρ, Llama-3.1 shows negative ρ, and Phi-3.5 is non-significant. When the signal is this unstable, direction reversal has minimal impact and small fluctuations can favor either direction.

B.3 VOC Non-Negativity Scope

In the classical VOC framework (Russell and Wefald, 1991), $\mathrm{VOC}(T, s) = \mathbb{E}[\max(V(a_T), V(a_{\mathrm{base}}))] - V(a_{\mathrm{base}}) \geq 0$ because the agent can *disregard* the computation result if unfavorable. Our setting violates this: when the agent uses an evaluator/optimizer T, the evaluator's assessment *is* the agent's decision. The agent cannot separately evaluate the evaluator's output because the evaluator *is* the evaluation mechanism.

Under evaluator-executor identity, VOC can be negative: $\mathrm{VOC}(T, s) = \mathbb{E}[V(a_T(s))] - V(a_{\mathrm{base}}(s))$. If T systematically selects worse actions in Type I states, $\mathrm{VOC} < 0$ for those states. This explains why wrong-direction gating is not merely wasteful ($\mathrm{VOC} = 0$) but actively harmful ($\mathrm{VOC} < 0$).

C DIAL Implementation Details

This appendix expands on the implementation details deferred from Section 4.

Algorithm 1 DIAL: Direction-Informed Adaptive Learning

Require: Environment $\mathcal{E}$, agent policy π, optimizer T, universal feature set ϕ_{univ}, LLM $\mathcal{M}$, exploration budget N_{explore}, exploration rate $\varepsilon_{\text{explore}}$, gate threshold τ
Ensure: Deployed gate $g : \mathcal{S} \to \{0, 1\}$
// Phase 1: Explore
$\mathcal{D} \leftarrow \emptyset$
for episode $= 1, \ldots, N_{\text{explore}}$ **do**
  Run π in $\mathcal{E}$; at each step t, invoke T with probability $\varepsilon_{\text{explore}}$ independent of state
  Record (s_t, U_t) pairs into $\mathcal{D}$
end for
// Phase 2: Reason
$(p_{\mathcal{E}}, \mathcal{H}) \leftarrow \mathcal{M}(\text{summary}(\mathcal{D}))$ ▷ env profile + feature hypotheses
$\phi_{\text{cand}} \leftarrow \phi_{\text{univ}} \cup \text{extract}(\mathcal{H})$
Recompute $\phi_{\text{cand}}(s)$ for all $s \in \mathcal{D}$
// Phase 3: Learn
Standardize ϕ_{cand} on $\mathcal{D}$
Select C via 5-fold CV on held-out log-loss
Fit ℓ_1-regularized logistic regression: $\mathbf{w}, b \leftarrow \arg\min \mathcal{L}_{\text{logistic}}(\mathbf{w}, b; \mathcal{D}) + \frac{1}{C}\|\mathbf{w}\|_1$
$\phi \leftarrow$ features with non-zero w_i
$g(s) \leftarrow \mathbf{1}[\sigma(\mathbf{w}^\top \phi(s) + b) > \tau]$
return g

C.1 Hyperparameters and Training Details

Phase 1 runs for $N_{\text{explore}} = 50$ episodes with $\varepsilon_{\text{explore}} = 0.5$. Before fitting in Phase 3, all features in ϕ_{cand} are standardized to zero mean and unit variance on $\mathcal{D}$, ensuring that the interpretable weight signs discussed in Section 4.3 are not confounded by scale differences across heterogeneous features. We induce sparsity using ℓ_1-regularized logistic regression with regularization strength C selected by 5-fold cross-validation over $C \in \{0.01, 0.03, 0.1, 0.3, 1.0, 3.0, 10.0\}$, optimizing held-out log-loss. Features with zero coefficient after fitting are dropped from the deployed gate. The decision threshold τ in Eq. (5) is fixed to 0.5 for all environments.

C.2 Universal Feature Definitions

Table 6 defines the universal features available in every environment.

Table 6: Universal feature definitions.

Feature	Definition
`step_count`	Current step index t in the episode
`token_entropy`	Entropy of the next-token distribution
`action_entropy`	Entropy over candidate actions
`state_length`	Token count of the current state text

C.3 LLM Feature Layer

For environments where universal features may be inadequate, DIAL's optional LLM feature layer automates the proposal of task-specific signals. The LLM receives a structured summary of the exploration dataset $\mathcal{D}$, including: (1) aggregate statistics (total episodes, steps, optimizer trigger rate, overall positive-utility fraction), (2) per-step breakdowns of trigger rate and utility by step index, and (3) representative examples of states where the optimizer was triggered, paired with their utility labels (positive vs. negative).

Given this summary, the LLM is prompted to analyze patterns that distinguish positive-utility from negative-utility states and to produce a Python function that extracts exactly 5 task-specific features from the raw state text. Each feature must return a float value using only the Python standard library. The full prompt template is provided in §C.5.

The LLM-proposed features $\phi_{\text{LLM}}(s)$ are combined with the universal features to form the candidate pool $\phi_{\text{cand}}(s) = \phi_{\text{univ}}(s) \cup \phi_{\text{LLM}}(s)$. The ℓ_1-regularized gate (§4.3) then selects or discards each feature based on its predictive value for rollout utility. This two-stage design (LLM proposes, LASSO disposes) ensures that uninformative LLM features are automatically filtered out. As shown in Table 11, environments with richer task-specific structure (WebShop, TWExpress) retain more LLM features, while Plancraft retains none.

C.4 Online Adaptation

For drifting environments, DIAL can optionally refit during deployment using ε-greedy exploration ($\varepsilon : 0.1 \to 0$) and periodic retraining. During deployment, $\varepsilon_{\text{deploy}}$ decays from 0.1 to 0 over the first 100 episodes, and the gate is refit every $k = 30$ episodes using the accumulated ε-greedy samples. Refitting reuses the same solver configuration as the initial training, so no additional hyperparameter tuning is required at deployment time.

This mode is *not free*: ε-greedy overrides ignore the current gate, so in Type I environments a fraction of overridden steps will trigger harmful rollouts. For stationary environments, which are the setting of our main experiments, we recommend skipping online adaptation and relying on the offline-fitted gate.

C.5 LLM Reasoning Prompts

The full LLM prompt template is provided below. We use WebShop as the example, where LLM fea-

tures contribute most meaningfully.

LLM Feature Layer Prompt Template

```
You are an AI agent that has
been exploring an interactive
environment.  During exploration,
you sometimes performed ``rollouts''
(additional computation) to improve
your decisions.  Sometimes the
rollout was useful (positive
utility), sometimes it was not.
Your task:  Analyze the patterns in
your experience and write a Python
function that extracts features from
the state text that could predict
whether a rollout would be useful.
```

[Exploration statistics and representative examples provided]

```
Requirements:
- Return a dict with string keys and
float values.
- Use only Python standard library.
- Extract exactly 5 features based
on the patterns you observe.
- Focus on features that DIFFER
between useful and not-useful
rollout cases.
```

Figure 5: Prompt template used in the LLM feature layer (Phase 2) to propose task-specific features from exploration data.

D Experimental Setup

D.1 Environment Specifications

We evaluate on 6 environments spanning diverse agent tasks. Table 7 summarizes their base performance, optimizer headroom, and optimizer type.

HotpotQA (Yang et al., 2018). A multi-hop question answering task where the agent must retrieve and reason over multiple Wikipedia paragraphs to answer complex questions. The agent iteratively issues search queries and reads returned passages before producing a final answer.

WebShop (Yao et al., 2022). A web navigation task where the agent shops for a product matching a natural language instruction. The agent navigates a simulated e-commerce website by searching, clicking product links, and selecting attributes (size, color) before purchasing.

FEVER (Thorne et al., 2018). A fact verification task where the agent must classify a claim as *Supported*, *Refuted*, or *Not Enough Info* based on evidence retrieved from Wikipedia. The agent searches for relevant passages and reasons over retrieved evidence to produce a verdict.

APPS (Hendrycks et al., 2021). A code generation task where the agent solves competitive programming problems. Given a problem description with input/output examples, the agent generates a Python solution and can iteratively debug based on test case feedback.

TWExpress (Jansen and Côté, 2023). A text-based game where the agent navigates a simulated environment to complete household tasks (e.g., finding and heating objects). The agent issues natural language commands (go, open, take, use) and observes textual descriptions of the resulting state.

Plancraft (Dagan et al., 2024). A crafting planning task where the agent must produce a target item by executing a sequence of crafting recipes in a Minecraft-inspired environment. The agent selects recipes and manages inventory to reach the goal item.

These environments cover the full range of the Two-Source Model: FEVER and HotpotQA are dominated by Type I states (information poverty), WebShop and APPS are dominated by Type D states (decision difficulty), TWExpress is rollout-safe ($\Delta{=}{+}31.8$), and Plancraft is rollout-harmful ($\Delta{=}{-}7.0$).

Per-action evaluation generates $K{=}5$ candidate actions and selects the best; K-variant sampling generates $K{=}3$ code solutions and selects a passing one; LLM-Propose-K has the LLM propose $K{=}3$ action sequences, simulates each, and selects the highest-reward trajectory.

Table 7: Environment setup. Base/Always: SR without/with optimizer. Δ: rollout headroom.

Environment	Base SR	Always SR	Δ	Optimizer T
HotpotQA	49.0%	97.0%	+48.0	Per-action eval
WebShop	7.2%	43.0%	+35.8	LLM-Propose-K
FEVER	37.0%	99.8%	+62.8	Per-action eval
TWExpress	67.5%	99.3%	+31.8	Per-action eval
Plancraft	29.8%	22.8%	−7.0	Per-action eval
APPS	60.5%	79.5%	+19.0	K-variant sampling

D.2 Detailed Baseline Comparison

Table 8 provides a detailed comparison of all baselines evaluated in this paper. All six methods share a common assumption: a fixed mapping from signal value to compute need.

D.3 Computational Cost Breakdown

Table 9 breaks down the per-method cost structure. Cost is reported as ×base: total *deployment* tokens normalized by base_only tokens per episode. Calibration or exploration overhead is excluded for all

Table 8: Detailed comparison of baselines. All methods assume a fixed signal–utility direction. †Requires a separate labeled dataset for calibration.

Method	Signal	Direction	Granularity	Mechanism
CATTS (Lee et al., 2026)	Vote entropy + margin	High disagreement → trigger	Per-step	K=5 forward passes, arbiter on disagreement
SEAG† (Lee et al., 2025)	Mean token confidence	Low conf. → search	Per-problem	Platt scaling, search depth allocation
CaTS† (Huang et al., 2026)	Calibrated confidence	Low conf. → generate	Per-attempt	Self-calibration, Best-of-N early stopping
CoRefine† (Jin et al., 2026)	Conv1D confidence	Low conf. → refine	Per-problem	Halt / re-examine / redirect controller
s1_budget (Muennighoff et al., 2025)	Token budget	Fixed budget → stop	Per-problem	Budget-constrained generation
AUQ (Zhang et al., 2026)	Verbalized uncertainty	High uncert. → reflect	Per-step	Uncertainty-aware memory + reflection
DIAL	**Multi (auto)**	**Learned**	**Per-step**	Explore + reason + direction learning

methods, ensuring a fair comparison of runtime efficiency. This metric captures hidden per-step overhead that raw rollout counts miss: CATTS incurs K=5 forward passes every step regardless of the gating decision, and AUQ issues a confidence query every step. DIAL requires 50 exploration episodes for calibration but adds zero per-step gate overhead at deployment.

Table 9: Computational cost breakdown per method. Gate OH = additional LLM calls per step beyond the base proposer. Cost (×base) = total *deployment* tokens normalized by base_only tokens per episode, averaged across 6 environments. Calibration/exploration overhead is excluded for all methods. †Requires separate calibration data.

Method	**Gate OH/step**	**OH source**	**Avg ×b**
base_only	0	—	1.00
always_trigger	0	—	6.00
CATTS	+K=5 calls	Voting (fwd pass)	7.98
SEAG†	0	Reads logprob	6.37
CoRefine†	0	Reads entropy	6.33
CaTS†	0	Reads confidence	8.26
AUQ	+1 query	Confidence question	7.67
s1_budget	0	—	6.35
DIAL	**0**	**Learned gate**	**5.85**

E Full Results

Table 10 reports SR and Cost (×base) for all methods across 6 environments and 3 backbones. Cost includes all deployment LLM calls (base proposer, gate overhead, rollouts) normalized by base_only cost. Bounds comparison is in Table 2 (§5.2). Several patterns are consistent across backbones.

Qwen3-4B. DIAL achieves the highest SR in 4 of 6 environments (HotpotQA, WebShop, TWExpress, APPS). The ×base metric reveals hidden costs: CATTS on APPS costs 6.00×base (due to K=5 per-step voting) despite barely triggering rollouts, while DIAL costs only 2.61× at 12.2 pp higher SR. FEVER is the main exception: DIAL achieves the highest SR (49.8%) but at 16.51×base, the highest cost among all environments. This failure mode is analyzed in §6.

Phi-3.5-mini. Fixed-direction baselines degrade sharply on this backbone. On FEVER, all six baselines score at or below 20% (base: 7.2%), while DIAL reaches 20.5%. On WebShop, DIAL (57.3%) exceeds the best baseline (s1_budget, 43.0%) by 14.3 pp at lower cost (2.56× vs. 3.89×).

Llama-3.1-8B. Direction reversal interacts with model capacity. On TWExpress, CaTS collapses to near-base performance (35.8% vs. base 36.5%) because it rarely triggers, while DIAL achieves 94.8%. On FEVER, DIAL (54.7%) is competitive with CaTS (56.3%), unlike Qwen3 where FEVER is the weakest environment. On APPS, AUQ drops to 46.0% (below base 60.2%), demonstrating that wrong-direction gating actively harms performance on this backbone.

F Signal and Feature Analyses

This appendix collects empirical analyses of signal identity and signal sufficiency referenced in Section 3.1.

Table 10: Full results across 6 environments and 3 backbones. SR (%) and Cost (×base, total deployment tokens normalized by base_only). Best and second-best SR per environment. †: methods requiring calibration data. Cost includes all LLM calls: base proposer, gate overhead (CATTS K=5 voting, AUQ confidence query), and rollout calls.

Method	HotpotQA		WebShop		FEVER		TWExpress		Plancraft		APPS	
	SR↑	Cost↓	SR↑	Cost↓	SR↑	Cost↓	SR↑	Cost↓	SR↑	Cost↓	SR↑	Cost↓
Qwen3-4B-Instruct												
CaTS†	93.2	10.54	30.5	3.45	48.2	25.48	96.7	3.37	20.3	4.14	66.2	2.61
SEAG†	67.5	9.34	28.0	2.84	47.3	17.19	97.3	2.60	22.8	3.52	66.0	2.75
CoRefine†	68.2	9.28	27.5	2.78	47.8	17.19	97.5	2.57	20.8	3.44	67.5	2.70
CATTS	68.3	10.53	16.0	5.55	32.2	16.46	97.5	2.83	23.0	6.53	60.8	6.00
AUQ	95.0	10.29	35.7	5.83	38.7	18.04	95.5	1.95	22.2	6.64	64.7	3.28
s1_budget	94.0	8.27	17.8	2.91	44.2	19.21	95.0	2.02	16.3	3.02	69.0	2.65
DIAL	**95.2**	8.02	**43.8**	2.50	**49.8**	16.51	**99.0**	1.81	**23.3**	3.63	**73.0**	2.61
Phi-3.5-mini-Instruct												
CaTS†	68.3	5.86	41.7	2.52	19.8	14.51	68.2	1.00	13.5	1.44	30.5	3.55
SEAG†	88.3	7.82	36.5	2.43	13.5	5.61	92.5	2.58	14.7	2.29	27.8	2.59
CoRefine†	87.7	7.83	35.8	2.44	13.7	5.73	92.7	2.56	15.2	2.24	28.5	2.60
CATTS	42.0	7.98	28.7	4.86	12.7	7.65	86.7	4.00	13.5	5.96	27.7	6.10
AUQ	31.2	5.47	37.0	3.11	8.5	3.18	92.5	1.82	12.7	2.24	27.2	2.03
s1_budget	91.2	5.26	43.0	3.89	20.0	8.45	93.7	2.05	13.7	1.73	34.5	2.93
DIAL	**92.3**	5.38	**57.3**	2.56	**20.5**	7.44	**96.7**	1.83	**16.8**	1.33	**36.8**	2.87
Llama-3.1-8B-Instruct												
CaTS†	94.2	7.21	36.0	2.51	**56.3**	14.82	35.8	3.00	22.8	3.42	55.2	4.81
SEAG†	94.0	9.82	27.5	1.86	53.5	12.57	76.5	3.85	21.8	2.70	57.2	4.41
CoRefine†	94.2	9.42	27.5	1.82	53.5	12.64	76.0	3.56	21.5	2.69	57.2	4.37
CATTS	94.0	11.11	11.3	4.74	14.2	17.00	49.3	4.11	27.0	7.31	52.3	6.70
AUQ	95.0	10.54	37.8	4.54	52.8	25.08	63.2	3.58	27.0	8.21	46.0	6.11
s1_budget	94.5	8.08	11.2	1.87	13.8	8.56	76.1	3.05	26.3	2.53	52.5	2.33
DIAL	**95.5**	7.07	**41.7**	1.68	54.7	18.69	**94.8**	2.55	**27.0**	2.49	**59.7**	4.29

F.1 Per-Environment Feature Selection

The LASSO coefficient matrix shows that `step_count` is informative in 4 of 5 environments with significant signal. WebShop is the exception, relying on `num_available_actions` and task-specific features (`price_mentioned`, `action_is_click`) reflecting its choice-dominated structure.

F.2 LLM-Generated Features

Table 11 reports the features generated by the LLM reasoning step (Phase 2) for each environment. The LLM receives exploration statistics and representative positive/negative examples, then produces a Python function extracting 5 task-specific features (the full prompt template is in §C.5). LASSO subsequently selects or discards each feature based on its predictive value for rollout utility.

Environments with rich task-specific structure benefit most from LLM features: WebShop's choice-dominated interface is well captured by `price_mentioned` and `action_is_click`, while TWExpress's exploration mechanics are reflected in `closed_ratio` and `action_look_around`. In contrast, Plancraft's rollout-harmful nature means no LLM feature improves prediction—LASSO correctly filters all five proposals. This pattern aligns with the LLM ablation results (Table 4): the environment where LLM features contribute most to SR (FEVER, +9.1 pp) is precisely where the LLM discovers unique signals (`has_claim`) that the universal feature pool lacks.

Table 11: LLM-generated features per environment. Each environment receives 5 LLM-proposed features; LASSO selects those with non-zero coefficients. Environments with richer task-specific structure (WebShop, TWExpress) retain more LLM features.

Environment	Sel./Gen.	LASSO-Selected LLM Features
WebShop	4/5	`price_mentioned`, `action_is_click`, `step_early`, `instruction_keyword_count`
TWExpress	4/5	`text_length`, `closed_ratio`, `action_look_around`, `already_open`
HotpotQA	3/5	`has_question`, `entity_count`, `fact_keyword_count`
FEVER	2/5	`text_length_normalized`, `has_claim`
APPS	1/5	`action_type`
Plancraft	0/5	(none selected)

Figure 6 shows the complete LLM-generated feature extraction function for WebShop (seed 42), the environment with the most LLM features retained.

LLM Output for WebShop

```
def extract_features(state_text, step_count,
    action_text):
    import re
    features = {}
    # Instruction keyword density
    keywords = ["gluten free", "protein",
        "price lower than", "size", "color"]
    matches = sum(1 for kw in keywords
        if kw in state_text.lower())
    features['instruction_keyword_count'] = (
        float(matches) / len(keywords))
    # Early-step indicator
    features['step_early'] = (
        1.0 if step_count <= 2 else 0.0)
    # Click vs. search action
    features['action_is_click'] = (
        1.0 if action_text.startswith("click[")
        else 0.0)
    # Price constraint presence
    price_matches = len(re.findall(
        r"price lower than|price:|\$",
        state_text, re.IGNORECASE))
    features['price_mentioned'] = (
        float(price_matches) / 5.0)
    # Filter complexity
    filter_kws = ["size","color","flavor",
        "price","rating"]
    features['filter_count'] = sum(
        1 for kw in filter_kws
        if kw in state_text.lower()) / 5.0
    return features
```

Figure 6: Complete LLM-generated feature extraction function for WebShop (seed 42), the environment with the most LLM features retained by LASSO.

F.3 AUC Hierarchy: Single vs. Multi-Signal Gates

Table 12 compares gate families on held-out data. Single entropy is near chance (AUC≈0.50) in every environment; multi-signal logistic gates reach 0.74–0.92; and hidden-state probes reach 0.79–0.99. The bottleneck is signal *combination*, not gate *capacity*.

Table 12: AUC by gate family. Single entropy is near chance; multi-signal gates close most of the gap to hidden-state probes. FEVER and TWExpress are omitted because their short episodes yield insufficient held-out data for reliable AUC estimation.

Gate family	HotpotQA	APPS	WebShop	Plancraft
Single entropy	0.50	0.56	0.50	0.50
Best single signal	0.78	0.78	0.90	0.74
Multi-signal LR	0.85	0.76	0.92	0.74
Hidden-state probe	0.87	0.79	0.99	0.95

G Robustness and Additional Experiments

This appendix collects Two-Source Model verification data, robustness checks for direction reversal, and additional experimental analyses.

G.1 Temporal Dynamics (P1)

Table 13 reports the full early/late ρ data referenced in Figure 4. The Two-Source Model predicts that $\rho(\text{entropy}, U)$ should decrease from early to late steps within each episode, as late-step uncertainty increasingly reflects the residual Type I component after information-gathering attempts.

Table 13: Temporal shift in $\rho(\text{entropy}, U)$. All environments show ρ shifting more negative from early to late steps, consistent with P1. Plancraft omitted ($|\rho|$=0.016, too weak for meaningful comparison).

Env	Phase	ρ	95% CI	n	p	Shift
HotpotQA	early	−0.176	[−0.260, −0.091]	509	<0.001	Δ=−0.261
	late	−0.437	[−0.500, −0.370]	699	<0.001	
APPS	early	+0.102	[−0.013, +0.209]	290	0.084	Δ=−0.246
	late	−0.144	[−0.281, +0.001]	199	0.043	
WebShop	early	+0.285	[+0.222, +0.346]	600	<0.001	Δ=−0.291
	late	−0.006	n.s.	473	0.895	
FEVER	early	+0.054	n.s.	200	0.446	Δ=+0.024
	late	+0.078	n.s.	82	0.486	
TWExpress	early	+0.161	—	422	<0.001	Δ=−0.153
	late	+0.008	n.s.	376	0.877	

The prediction holds across 4 of 5 testable environments: HotpotQA shows the strongest

shift ($\Delta{=}-0.261$), with late-step entropy reflecting deeper information poverty after failed evidence retrieval. WebShop's strong positive early-phase signal ($\rho{=}+0.285$) vanishes in the late phase ($\rho{=}-0.006$, n.s.), consistent with the choice-dominated structure being resolved early. In APPS, ρ reverses from marginally positive ($+0.102$) to negative (-0.144), as late-step decision complexity gives way to residual information poverty.

FEVER is the exception: both phases show weak, non-significant positive ρ, consistent with FEVER's short episodes (median 8 steps) concentrating most rollout value at step 0–1, leaving insufficient late-step variation for a meaningful shift.

G.2 Signal Identity Alignment (P3)

Table 14 reports the strongest signal per environment and its interpretation under the Two-Source Model. In Type I environments, information-sufficiency proxies (`step_count`) dominate; in mixed/Type D environments, decision-complexity proxies (`num_available_actions`) dominate.

Table 14: P3 Signal identity alignment. The strongest signal in each environment matches the Two-Source Model prediction: information-sufficiency proxies dominate in Type I environments; decision-complexity proxies dominate in mixed/Type D.

Env	Type	Strongest Signal	$\lvert\rho\rvert$	Interpretation
HotpotQA	Info-Poverty	step_count	0.494	Info accumulation
FEVER	Info-Poverty	step_count	0.619	Info accumulation
TWExpress	Info-Poverty	step_count	0.477	Exploration depth
WebShop	Mixed	num_avail_actions	0.444	Decision-space size
APPS	Decision-Diff	step_count	0.339	Debugging depth
Plancraft	Weak (harmful)	has_output	0.162	Rollout harmful

G.3 Multi-Backbone Verification

We validate our core findings on three backbones from three vendors: Qwen3-4B-Instruct (Alibaba, 4B), Phi-3.5-mini-instruct (Microsoft, 3.8B), and Llama-3.1-8B-Instruct (Meta, 8B) across all 6 environments. Table 10 reports full results.

Figure 7 visualizes the direction variation, with gold borders marking sign flips. On Phi-3.5 FEVER: most fixed-direction baselines score near or below base_only (7.2% base $\rightarrow$ 8.5–23%), demonstrating that the fixed-direction assumption is brittle when the model changes (Figure 8).

G.4 Stratified Analysis: Controlling for Trajectory Length

Trajectory length varies across environments and correlates with both entropy and utility. To rule out length as a confounder, we stratify by step count and recompute $\rho(\text{entropy}, U)$ within each stratum. Direction reversal persists within trajectory-length strata where utility variance is non-zero. In HotpotQA, ρ remains negative across all three strata (-0.18/-0.46/-0.42). The reversal is not an artifact of trajectory length.

G.5 Regularizer Ablation

The signed-weight diagnostic of §4.3 and DIAL's robustness to noisy LLM-proposed features (§4.2) both rely on the *exact* sparsity induced by the ℓ_1 penalty in Eq. (4). To verify that ℓ_1 specifically— rather than any shrinkage or selection mechanism—is responsible for these properties, we ablate the regularizer while holding all other pipeline components fixed: same exploration dataset $\mathcal{D}$, same candidate feature pool ϕ_{cand}, same 5-fold CV protocol for hyperparameters, same threshold $\tau{=}0.5$. We compare five variants:

- **ℓ_1-logistic** (DIAL default): joint sparsity and shrinkage.
- **ℓ_2-logistic** (Ridge): shrinkage only; all features retained with small non-zero weights.
- **No regularization**: plain logistic fit on ϕ_{cand}.
- **Elastic Net** ($\alpha{=}0.5$, mixing ℓ_1 and ℓ_2).
- **MI top-k + unregularized**: hard feature selection by mutual information (top-3), then unregularized logistic on the retained features. This variant isolates *selection* from *shrinkage*.

Table 15 reports SR and the mean number of non-zero fitted weights across three environments spanning signal strength: HotpotQA ($|\rho|{=}0.494$, strong), WebShop ($|\rho|{=}0.444$, moderate), and Plancraft ($|\rho|{=}0.016$, weak).
Note. ℓ_1 #nz values are measured across three seeds of DIAL's production runs; the per-environment mean reveals adaptive sparsity: under strong signal (HotpotQA, WebShop) DIAL retains $\approx$7–8 features, whereas under weak signal (Plancraft) ℓ_1 collapses the gate to $\approx$2 features, absorbing the remainder into the bias and yielding a near-constant trigger probability consistent with the low rollout headroom of that environment (Figure 9).

The comparison isolates three hypotheses underlying DIAL's regularizer choice.

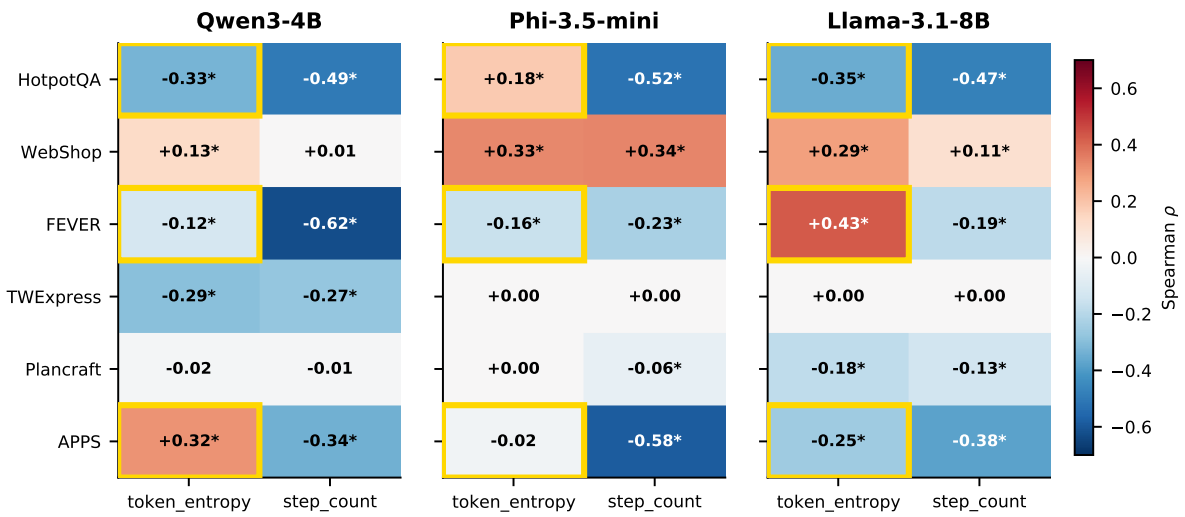

Figure 7: Multi-backbone signal direction heatmap. Gold borders mark cells where the sign of ρ flips across backbones with significant correlations. Entropy direction is model-dependent; structural signals (step_count) are more stable.

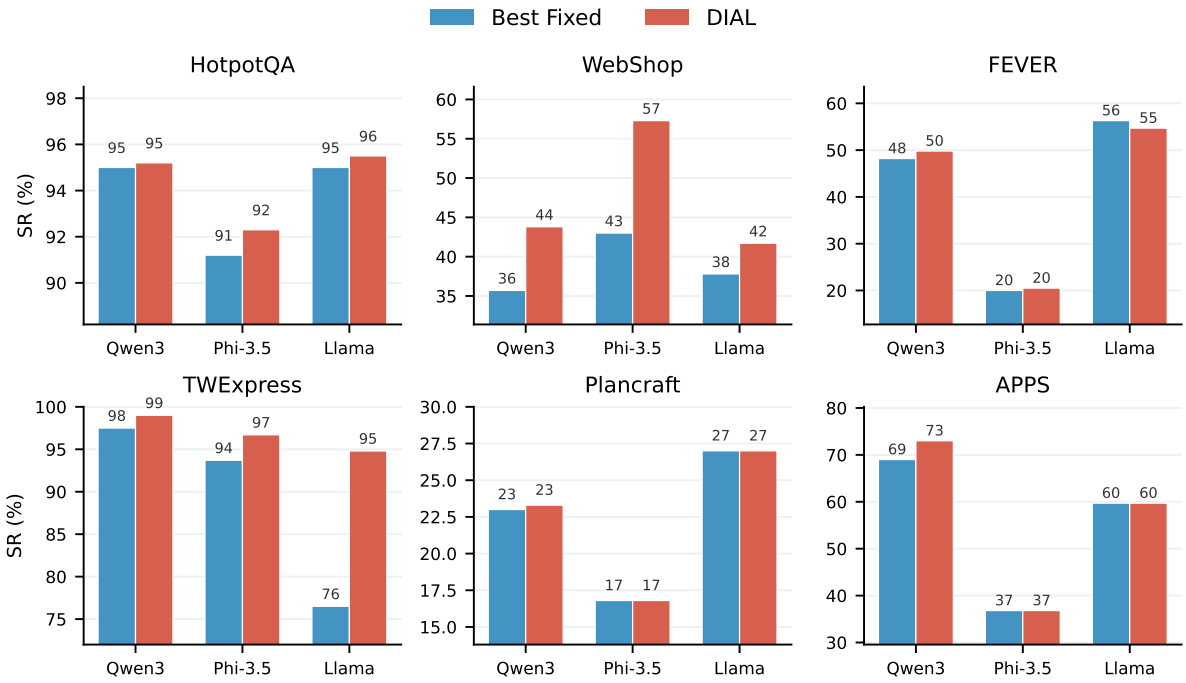

Figure 8: Cross-backbone comparison: DIAL (red) vs. best fixed-direction baseline (blue) across 3 backbones. Fixed baselines show large SR variance across backbones; DIAL adapts robustly.

ℓ_1 vs. ℓ_2: sparsity is required for diagnosis, not just for predictive performance. Both penalties shrink noisy coefficients toward zero, so on strong-signal environments their gating SRs are expected to be comparable. The critical distinction lies in interpretability: under ℓ_2, every feature retains a small non-zero weight, collapsing the clean {informative, uninformative} partition of §4.3 into a continuous ranking. The signed-weight summary no longer reads off which signals the environment treats as informative—it merely ranks them. Reports of "non-zero weights per environment" in the main text (e.g., the direction analyses of Table 3) are only interpretable under exact sparsity.

ℓ_1 vs. unregularized logistic: robustness under weak signals. On weak-signal environ-

Table 15: Regularizer ablation on three environments ordered by signal strength. SR (%); "#nz" is the mean number of non-zero fitted weights across three seeds. The candidate pool ϕ_{cand} contains 31 features (5 universal $+3$ derived $+20$ hidden-state PCA $+3$ auto-extracted); adding the LLM feature layer brings the pool to $\approx$36. ℓ_1 is the only variant that achieves competitive SR *and* exact sparsity required by the signed-weight diagnostic.

	HotpotQA $\lvert\rho\rvert$=0.494		WebShop $\lvert\rho\rvert$=0.444		Plancraft $\lvert\rho\rvert$=0.016	
Regularizer	SR	#nz	SR	#nz	SR	#nz
ℓ_1 (DIAL)	95.2	7.3	43.8	8.3	23.3	1.7
ℓ_2	TBD	36	TBD	36	TBD	36
No regularization	TBD	36	TBD	36	TBD	36
Elastic Net	TBD	TBD	TBD	TBD	TBD	TBD
MI top-3 + unreg.	TBD	3	TBD	3	TBD	3

ments like Plancraft, the unregularized fit is expected to latch onto sample-level noise: with $|\rho|\approx 0.02$, any 250-sample fit without penalty will assign large weights to whichever feature happens to pseudo-correlate on the training split. ℓ_1's shrinkage absorbs this noise into the bias term, producing a gate that triggers conservatively—consistent with the observed low trigger rate in Plancraft (Figure 9).

ℓ_1 vs. MI top-k + unregularized: joint selection+shrinkage vs. selection alone. Hard feature selection by mutual information separates the two roles ℓ_1 plays. The resulting gate is expected to recover most of ℓ_1's SR on strong-signal environments, but exhibits two practical drawbacks: (i) k must be tuned per environment rather than emerging from CV over a single hyperparameter C, reintroducing the per-environment engineering cost that DIAL aims to eliminate; and (ii) the retained features lose the shrinkage calibration that stabilizes weights near the decision boundary, making the gate more sensitive to τ.

Summary. The three comparisons establish that ℓ_1 is not interchangeable with other shrinkage or selection schemes. It is the minimal choice that simultaneously delivers competitive SR, the *exact* sparsity required by the signed-weight diagnostic, and robustness to noisy feature proposals from the optional LLM layer.

G.6 Gate Complexity Ablation

To verify that the bottleneck is direction rather than gate complexity, we compare gate architectures with correct vs. wrong direction (Table 16).

Table 16: Gate complexity ablation on HotpotQA. Direction matters more than model capacity: a logistic gate with correct direction outperforms an MLP with wrong direction by 49.9 pp.

Gate	Direction	SR (%)	Δ
Logistic (5 feat)	Correct	95.2	+46.2
MLP (5 feat)	Correct	$\sim$95	$\sim$+46
Hidden state (2560-d)	Correct	$\sim$95	$\sim$+46
Logistic (5 feat)	Wrong	62.0	+13.0
MLP (5 feat)	Wrong	45.3	-3.7
No gate (base)	n/a	49.0	0

With correct direction, all architectures achieve $\sim$95% SR; the jump from logistic to MLP to hidden-state probe adds $<$1 pp. With wrong direction, higher capacity makes performance *worse* (MLP 45.3% $<$ logistic 62.0%), because the more powerful gate more precisely targets harmful states. This confirms that signal direction consistently outweighs gate complexity, and explains why DIAL's minimal gate design suffices.

G.7 Environment-Adaptive Trigger Behavior

Beyond direction and feature selection, DIAL's gating magnitude also adapts to each environment without explicit programming. Figure 9 shows the per-step trigger rate across all 6 environments, ordered by rollout headroom Δ. In rollout-safe TWExpress (Δ=+31.8), DIAL triggers aggressively (RR = 73%), consistent with the high headroom. In rollout-harmful Plancraft (Δ=-7.0), the trigger rate decays from 49% at step 0 to $<$20% in later steps, as the gate learns that late-step rollouts are increasingly harmful. WebShop and APPS show moderate, stable trigger rates. This full behavioral range emerges from the per-environment signed weights fitted in §4.3: the gate does not need an explicit headroom estimate, because signal direction already encodes whether the environment rewards or penalizes additional computation.

H Broader Impact

This work addresses computational efficiency in LLM agent deployment. By reducing unnecessary optimizer invocations, DIAL lowers the energy and cost footprint of test-time compute without sacrificing task performance. The prescriptive framework (characterize information structure before designing gates) may help practitioners avoid deploying fixed-direction methods that actively harm performance in mismatched environ-

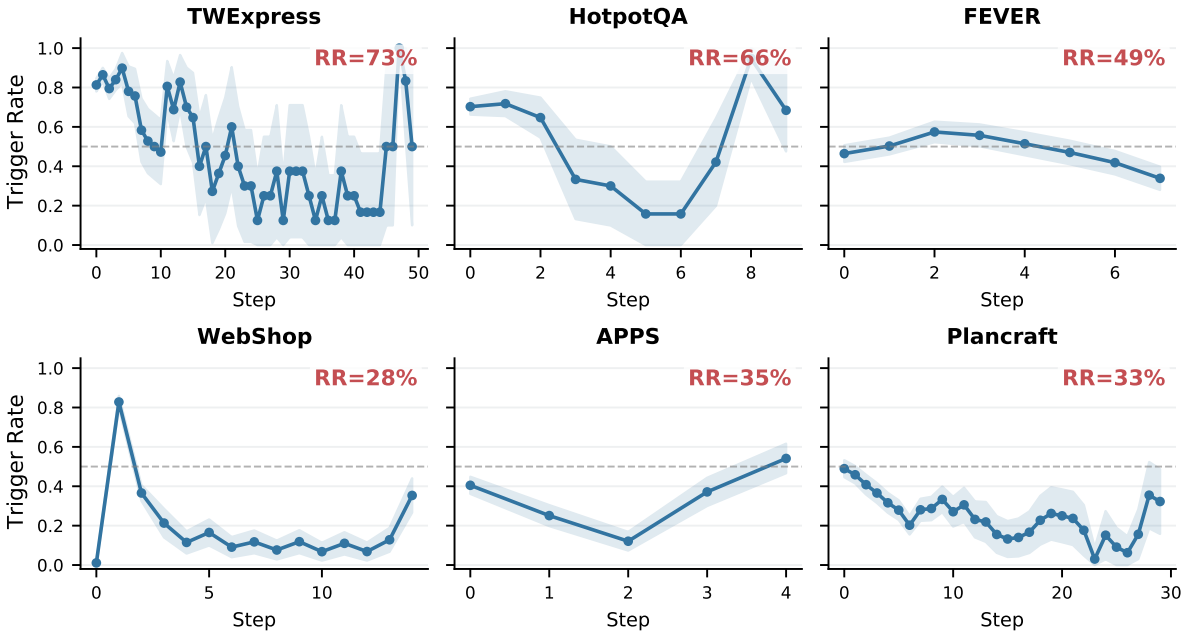

Figure 9: Environment-adaptive trigger behavior. Panels ordered by rollout headroom Δ. Solid line: per-step trigger rate with 95% CI band. Dashed gray: 50% reference. RR: overall trigger rate. DIAL's trigger rate ranges from 73% (TWExpress) to <20% (late-step Plancraft), emerging from direction learning alone.

ments. We do not foresee direct negative societal impacts beyond those inherent to the LLM agents themselves.